

A GPU-Batched Time-Evolution Backend for `tkwant`

Yongho Park

September 22, 2026

Abstract

`tkwant` [1] propagates each one-body scattering state of a many-body wavefunction sequentially on the CPU. We present a GPU-batched alternative: all locally held one-body states are stacked into a single matrix and integrated with a custom batched Dormand–Prince (`dopri5`) solver, replacing N_{states} sequential CPU ODE solves with one batched GPU solve. For a purely sinusoidal (monochromatic) drive, the tight-binding Hamiltonian has an exact closed form in $\cos(\omega t)$ and $\sin(\omega t)$, so no per-step Hamiltonian reassembly is needed at all. General drives instead use a numeric fallback that reassembles the Hamiltonian at a controllable rate. For drives that are both ultrashort and spatially local, such as a voltage pulse through a small quantum dot, evaluating the local perturbation directly at every integration substep and bounding the adaptive step size removes the reconstruction cost altogether. On a synthetic pulse system and against a fair MPI-parallel CPU baseline, the GPU reaches $8.9\times$ at $N_{\text{states}} = 700$. This speedup comes from the number of `evolve()` calls the CPU needs for a correct answer (about 12,000 checkpoints across the pulse, against one GPU call), not from the system size. On the quantum-dot system of a published pulse study, where both sides make the same 5001 calls, a 24-rank CPU run was $1.9\times$ faster than the GPU for a moderate pulse, and the narrowest published pulse could not be completed by native CPU at all. The method is validated against native CPU `tkwant` on six structurally distinct Hamiltonian classes (relative error 10^{-7} – 10^{-3}) and benchmarked against `tkwant`’s own MPI-parallel CPU baseline: the GPU loses at small batch size and wins by up to $75\times$ at production scale. We further extend the method to `tkwant`’s self-consistent (mean-field) solver and validate it against native CPU evolution (relative error 8×10^{-4}).

1 Introduction

`tkwant` [1] extends `kwant` [2] to real-time, real-space quantum transport. Leads are represented by a complex absorbing potential (CAP) rather than by mode matching, and the scattering problem is solved by explicit time propagation of a wavefunction correction on top of a static scattering state [3]. No GPU backend exists for `tkwant`. A 2016 `kwant`-discuss thread on GPU acceleration identified the bottleneck for *static* `kwant` as lead-mode computation, a generalized Schur (QZ) decomposition with no practical GPU implementation, together with the sparse direct solve. Neither has since been ported, and that effort targeted `kwant`’s mode-matching formalism, a different computational problem from `tkwant`’s.

`tkwant` avoids mode matching entirely. The CAP/psibar formalism of Ref. [3] turns the scattering problem into an ODE integration problem rather than a generalized eigenproblem, and ODE integration batches trivially across many one-body states on a GPU. It therefore sidesteps the bottleneck that stalled GPU work on `kwant`’s static side in 2016, which motivates the backend described here.

2 tkwant’s current time-evolution process

A single one-body scattering state in `tkwant` (`onebody.WaveFunction`) solves

$$i \partial_t \bar{\psi} = (H_0 + W(t) - E) \bar{\psi} + W(t) \psi_{st}, \quad \psi = (\bar{\psi} + \psi_{st}) e^{-iE(t-t_0)}, \quad (1)$$

where H_0 is the static part of the tight-binding Hamiltonian, $W(t)$ the time-dependent perturbation, E the energy of the incoming scattering state, and ψ_{st} the time-independent scattering-state solution of $H_0 \psi_{st} = E \psi_{st}$ at $t = t_0$ [3]. The right-hand side is evaluated by a *kernel* object (`onebody.kernels`). The state is advanced by a *solver* object, by default an adaptive Runge–Kutta integrator.

A many-body state (`manybody.WaveFunction`) holds N_{states} such one-body tasks, one per (lead, mode, energy) triple contributing to the occupied scattering states, each carrying a quadrature weight set by the lead occupation. Tasks are distributed round-robin across MPI ranks (`tkwant.mpi.DistributedDict`). `evolve()` advances every locally held one-body state *sequentially*, one `scipy`-based ODE solve at a time, and `evaluate()` forms the quadrature-weighted sum of a `kwant.operator`-style observable over all tasks, MPI-reduced across ranks. `tkwant` therefore already parallelizes *across* one-body states via MPI, but it does not accelerate any *single* state’s ODE solve, nor does it batch multiple states’ solves on one device.

3 GPU solver

3.1 Batched ODE reformulation

Every one-body task sees the same time-dependent effective Hamiltonian $H_{\text{eff}}(t)$. Only ψ_{st} and E differ per task. Stacking the N_{states} $\bar{\psi}$ vectors as columns of a single matrix Ψ ($n_{\text{dof}} \times N_{\text{states}}$, n_{dof} = number of degrees of freedom, as in the tables below) turns Eq. (1) into one batched matrix ODE,

$$\frac{d\Psi}{dt} = -i H_{\text{eff}}(t) \Psi + i \Psi \text{diag}(\Delta E) + \text{src}(t), \quad (2)$$

integrated with a custom batched `dopri5` step (`BatchGPUSolver` in `gpu_solver_core.py`) so that a single batched sparse matrix–matrix product (cuSPARSE, NVIDIA’s sparse linear algebra library, via CuPy) replaces N_{states} sequential CPU sparse matrix–vector products at every substep.

3.2 Recovering H_{eff}

`tkwant` does not expose H_{eff} as a public matrix. Two paths are used, tried in order. (i) A fast path reads H_0 directly when the kernel happens to expose it as a plain attribute. (ii) Otherwise, for a compiled or opaque kernel that does not expose H_{eff} , it is recovered once by *column-probing* the kernel’s own right-hand-side function: column k of H_{eff} equals $i \times \text{rhs}(e_k)$ for the k -th unit vector e_k , reassembled into a sparse CSR matrix. The solver therefore never receives the Hamiltonian directly from `tkwant`’s internal system representation. It reconstructs it from the outside, through the same public interface available to any `tkwant` user.

Path (ii) is the one exercised in practice: `tkwant`’s default kernel, `kernels.Simple`, is a compiled, opaque `cdef` class that does not expose H_0 , so every benchmark in this note pays the n -probe cost once (the 379s production-scale reconstruction in Sec. 5 is this cost, not the ODE integration itself). `tkwant` does ship a pure-Python kernel, `kernels.Scipy`, whose `rhs().H0` attribute already exposes exactly this matrix, path (i) above. Exposing H_0 the same way on `kernels.Simple` would let every case use the fast path and remove the reconstruction cost entirely, an upstream change worth raising alongside the collaboration request (Sec. 6).

3.3 Exact monochromatic decomposition

When the time-dependent part of the Hamiltonian is purely sinusoidal at a single frequency ω , as for a monochromatic drive (an AC gate, a phonon at fixed ω , circularly or linearly polarized light, a single-tone Floquet drive), the tight-binding Hamiltonian admits an *exact* closed form,

$$H_{\text{TB}}(t) = H_{\text{TB}}(0) + (\cos(\omega t) - 1) H_A + \sin(\omega t) H_B, \quad (3)$$

with H_A, H_B obtained once from two extra Hamiltonian evaluations at $t = 0$. The source term $W(t)\psi_{st}$ is decomposed the same way. This eliminates per-step Hamiltonian reassembly entirely. The GPU right-hand side reduces to two scalar-weighted sparse matrix–vector products per substep, and the result is exact: the only discretization error left comes from the ODE integrator, not from the drive.

The constraint that $H(t)$ be expressible through $\cos(\omega t)$ and $\sin(\omega t)$ alone is formally narrow, but it covers most time-dependent perturbations of physical interest: single-frequency AC drives, monochromatic phonon or photon fields, and Floquet problems are all of this form by construction. What remains is genuinely non-monochromatic drives, such as pulses, quenches, and multi-tone drives, addressed next.

3.4 Non-monochromatic fallback

When the drive is not purely monochromatic, the solver falls back to periodic numeric reassembly of $H(t)$, still GPU-batched but without the closed-form speedup: $H(t)$ is either frozen over each reconstruction sub-interval $[t, t + \Delta t_{\text{rc}})$ (0th order) or linearly interpolated between the sub-interval’s endpoints (1st order, one extra Hamiltonian evaluation per interval). Convergence is confirmed to the expected order (frozen: 1st order in Δt_{rc} , linear: 2nd order). The relevant control parameter is $\Delta t_{\text{rc}}/T$ relative to the drive’s own characteristic timescale T , not an absolute value of Δt_{rc} : at production scale ($n_{\text{dof}} = 18,936$, $N_{\text{states}} = 30$, $\omega = 0.04$), $\Delta t_{\text{rc}}/T \in [0.0125, 0.05]$ gives 0.37–4.5% relative error at 35–52 $\times$ speedup over the fair CPU baseline (Sec. 5). The same adaptive-stepper caveat as Sec. 3.5 applies here too, independent of how local the drive is: a bound on the trial step size (`max_step`) is available as a general safety net whenever a drive varies faster than Δt_{rc} itself.

3.5 Ultrashort, spatially local drives

A drive confined to a small region (e.g. a single site or small dot) but much faster than the reconstruction interval needed elsewhere breaks the fallback above: Δt_{rc} would need to be pushed far below the drive’s own width, and the per-interval `hamiltonian_submatrix` cost then dominates completely (measured $\sim 1000\times$ slower than CPU at that extreme). Two changes fix this: evaluating the local perturbation via `kernels.PerturbationExtractor` (Sec. 3) at every `dopri5` substep instead of on a separate Δt_{rc} grid, cheap here since it touches only the few time-dependent matrix elements, and bounding the trial step size (`max_step`, as in `scipy.integrate.solve_ivp`) so the adaptive controller cannot silently step over a feature narrower than its current trial step.

We validated this on a synthetic Gaussian voltage pulse of width 0.05 against a hopping-set timescale of order 1 (system spec in Table 1). This pulse is genuinely narrow; the pulses of Refs. [5, 6] are not ($\sigma \gtrsim 6$ in their own units, Table 2). Median relative error is $2\text{--}5 \times 10^{-4}$ across $N_{\text{states}} = 10\text{--}200$ (accuracy was not re-checked at $N_{\text{states}} = 500, 700$ below, only wall-clock time was). At coarser checkpoint spacing than Table 1 uses, native tkwant’s adaptive stepper silently skips the pulse and returns a wrong answer, the same failure mode `max_step` is designed to prevent.

Against tkwant’s own MPI-parallel CPU baseline on a 24-core/32-thread workstation, one `manybody.WaveFunction` task per rank:

Table 1: Synthetic benchmark system (used in every table of this subsection). Full definition: `test_gpu_solver_core.py, make_1d_chain`.

Lattice	<code>kwant.lattice.square(a=1, norbs=1)</code> , single row (1D)				
Sites, hopping	40 sites, -1				
Chemical potential	0				
Drive	onsite, sites 16–24: $0.3 e^{-(t-10)^2/(2 \times 0.05^2)}$				
CAP boundary	$\tau_{\max} = 80$				
Quadrature	5th-order Gauss–Legendre per energy interval				
Evolved to	$t = 20$				
CPU checkpoints	$\Delta t = 1.0$; width/1000 within 0.3 of the pulse center				
GPU <code>max_step</code>	0.05 ($= \sigma$)				

N_{states}	ranks	CPU (multi-core) ¹	GPU (evolve)	result
10	10	1.46 s	5.10 s	CPU wins, $3.5\times$
50	24	6.06 s	5.27 s	GPU wins, $1.1\times$
100	24	9.64 s	5.52 s	GPU wins, $1.7\times$
200	24	16.87 s	5.61 s	GPU wins, $3.0\times$
500	24	38.88 s	6.19 s	GPU wins, $6.3\times$
700	24	55.10 s	6.19 s	GPU wins, $8.9\times$

This speedup is a call-count effect, not a system-size effect: this system’s 196 degrees of freedom are fewer than the 429 of Ref. [5]’s system (Table 2), where the GPU nonetheless loses (below). The result also does not transfer to Ref. [5]’s own system directly: its task count is set at runtime rather than swept by hand, and its pulses are wide enough that they do not need the fine checkpointing this benchmark required.

Table 2: The two systems referenced in this section.

	Synthetic (Tables above)	Ref. [5] (below)
Drive	onsite energy, Gaussian	hopping phase, $-e^{-i\phi(t)}$
Pulse width σ	0.05 (narrow)	$\gtrsim 6$ (not narrow)
Physical sites	40	5
n_{dof} (incl. CAP buffer)	196	429
N_{states}	swept, 10–700	set at runtime, $252 \rightarrow 1134$

Their published system, timed end to end ($\bar{n}$ is their pulse-area parameter):

The 252-task GPU row’s larger error is a quadrature effect (too few energy points), not a time-integration error. We do not claim a speedup on this system: the GPU only beats the single-core native run. For the narrowest published pulse ($\gamma_{\text{eff}} = 0.0002$) no CPU baseline exists at all: native tkwant did not complete it (below), so the only completed run used the CPU-to-GPU handoff described next.

As a preliminary check, we compared directly against published data (CPU-to-GPU handoff as above) across three parameter regimes. This covers one drive amplitude ($\bar{n} = 1$) and three of the paper’s roughly two dozen parameter combinations: a first pass, not a full reproduction.

The moderate-regime error is the same 252-task quadrature effect noted above, and drops to

Regime	$\gamma_{\text{eff}} = 0.02, \tau_p = 10, \bar{n} = 1$
Output times	5001
Native task count	252 $\rightarrow$ 1134 (grows via <code>refine_intervals()</code>)
GPU task list	fixed from $t = 0$: either the initial 252 or final 1134
GPU <code>max_step</code>	$\sigma/5 \approx 1.2, \sigma = \tau_p/(2\sqrt{\ln 2})$

Run	ranks	wall time	N_{states}	rel. error ²	vs. 24-rank CPU
Native CPU, single core	1	481.5 s	252 $\rightarrow$ 1134	2×10^{-9}	4.0 $\times$ slower
Native CPU, MPI	24	119.5 s	252 $\rightarrow$ 1134	(same algorithm)	reference
GPU, initial tasks	–	185.1 s	252 fixed	1.8×10^{-3}	1.5 $\times$ slower
GPU, final tasks	–	233 s	1134 fixed	2×10^{-8}	1.9 $\times$ slower
With $n(t), j(t)$ evaluated every output time, as in the authors’ own loop:					
Native CPU, MPI	24	127.4 s	252 $\rightarrow$ 1134	–	reference
GPU, final tasks	–	328 s	1134 fixed	–	2.6 $\times$ slower

2×10^{-8} with all 1134 tasks. The narrowest-pulse regime is the hardest case for two independent reasons. First, its small physical signal (footnote 6) makes the same absolute error a much larger relative one than in the other two regimes. Second, native tkwant’s own `refine_intervals` becomes pathologically slow in this same regime, independent of our extension:

This looks like a weakness of adaptive quadrature refinement under an extreme pulse-width-to-output-grid ratio rather than a bug, and is a further practical case for a fixed task list plus `max_step`. The moderate regime is therefore our best-validated case against the published data, not the narrowest pulse, which is the harder case for both reasons above.

This speedup requires the drive to be spatially local. A drive that is both spatially broad and ultrashort in time is not addressed by either path in this note, and we do not have a validated answer for that combination.

4 Validation

The generic core (`gpu_solver_core.py`) touches only tkwant’s public per-state API (`state.kernel.rhs()`, `state.psibar`, `state.psi_st`) and `fsys.hamiltonian_submatrix()`. It contains no reference to orbital count, lattice symmetry, or dimensionality. It was validated against native CPU tkwant on six structurally distinct systems, comparing both the raw $\bar{\psi}$ trajectory and, on one system, the physical observables (`kwant.operator.Current`, `kwant.operator.Density`):

Physical-observable agreement (1D chain, current and density operators) tracks the raw $\bar{\psi}$ -level accuracy at 5×10^{-9} – 1.6×10^{-7} , confirming that downstream observables inherit the ODE-level accuracy. The BdG case matters most for the genericity claim. Column-probing and the exact-Fourier decomposition never special-case what a matrix element represents physically (onsite vs. hopping, particle vs. hole channel); they require only that $H_{\text{TB}}(t)$ be affine in $\cos(\omega t)$ and $\sin(\omega t)$ as an abstract sparse matrix. Spin-orbit coupling, disorder, and multi-tone (more than one simultaneous drive frequency) Hamiltonians have not yet been tested. Chiral phonons, an actively studied drive, are a single-frequency case already covered by Sec. 3.3 [4].

Regime	$(\gamma_{\text{eff}}, \tau_p)$	n_{dof}^3	N_{states}	rel. err. $n(t)^4$	rel. err. $j(t)^5$
Narrowest published pulse ⁶	(0.0002, 10)	5105	546 $\rightarrow$ 714	20%	17%
Moderate	(0.02, 10)	429	252 $\rightarrow$ 1134	0.17%	0.16%
Broad, gentle	(0.02, 200)	553	252 $\rightarrow$ 273	$\sim 10^{-8}\%$	$\sim 10^{-8}\%$

Table 3: `refine_intervals` cost on the exact published, narrowest-pulse system (single CPU process).

Output step i	N_{states}	per-step cost
480	546	0.15 s
500	546	4.9 s
513	672	6.1 s
548	714	7.8 s
596	714 $\rightarrow$ 735	stalled, still running after hours (killed)

5 Benchmarks

`tkwant` already distributes its N_{states} one-body tasks across MPI ranks natively (Sec. 2), so `evolve()` under `mpiexec -n P` runs P -way parallel with no code changes. Comparing single-GPU against single-core CPU is therefore not a fair baseline. All comparisons below use `tkwant`’s own MPI-parallel CPU evolution (10–24 ranks on a 24-core/32-thread workstation) as the reference:

This crossover holds consistently, not as a single cherry-picked point. At small batch size ($N_{\text{states}} \sim 10$), GPU per-step dispatch overhead dominates and the CPU wins by more than an order of magnitude. The GPU overtakes it from $N_{\text{states}} \sim 130$ –220 onward, and the margin grows with both N_{states} and the integration length t_{max} . At production scale, the actual physics model this solver was built for ($n_{\text{dof}} = 18,936$, $N_{\text{states}} = 30$, $t = 10$ –500), the long integration window dominates over the modest batch size: 24-core CPU evolution takes 1885.8 s, single-GPU evolution takes 25.0 s (plus a one-time, cacheable 379 s Hamiltonian reconstruction), a $75.4\times$ speedup, independently confirmed accurate to 0.105% against the same CPU reference (Sec. 4). The controlling variables are $(n_{\text{dof}}, N_{\text{states}}, t_{\text{max}})$, not lattice type. The GPU wins beyond a problem-size/integration-length threshold, with the margin growing from there, rather than being faster unconditionally.

6 Conclusion and outlook

The batching, H_{eff} -reconstruction, and exact-decomposition core makes no assumption about lattice geometry, orbital count, or dimensionality, and is validated across six structurally unrelated Hamiltonian classes, with a benchmarked speedup regime against `tkwant`’s own MPI CPU baseline. Feedback from the `tkwant` maintainers on whether any part of this is worth upstreaming would be welcome.

We have also implemented and validated an extension to `tkwant`’s self-consistent (mean-field) solver, `interaction.SelfConsistentState`. There, a mean-field potential $q(t)$ is added on top of $W(t)$ through `WaveFunction.add_perturbation` as a diagonal shift, updated on a timescale τ coarser than the ODE substep, with $q(t)$ a low-order polynomial extrapolation of a mean-field operator’s expectation value evaluated at the previous τ -checkpoint. $q(t)$ is added to each one-body state’s *kernel* (`kernel.set_W`), not to `fsys`’s own onsite or hopping functions, so it is invisible to `fsys.hamiltonian_submatrix()`: neither the exact-Fourier nor the numeric-fallback path of Sec. 3

Lattice	orbitals/site	n_{dof}	N_{states}	rel. error
1-orbital 1D chain	1	868	10	10^{-7} – 10^{-8}
1-orbital 2D square ribbon	1	1728	128	2.2×10^{-7}
Honeycomb ribbon (2-sublattice)	1	3059	216	1.9×10^{-7}
p -orbital model (production scale)	3	18936	30	1.05×10^{-3}
BdG (particle–hole doubled), AC-Josephson pairing	2 (Nambu)	392	20	3 – 7×10^{-8}
3-lead system (2D slab, 3rd terminal on top edge)	1	4280	250	1.4 – 5.9×10^{-7}

Lattice	n_{dof}	N_{states}	CPU (multi-core)	GPU (evolve)	result
1D chain	196	10	0.149 s	6.349 s	CPU wins, $42.6 \times$
2D square ribbon (40×8)	1728	128	23.8 s	4.73 s	GPU wins, $5.0 \times$
Honeycomb ribbon (30×10)	3059	216	56.9 s	7.85 s	GPU wins, $7.2 \times$
Honeycomb ribbon (60×16)	2923	348	723.1 s	57.7 s	GPU wins, $12.5 \times$

sees it. Re-probing `state.kernel.rhs()` (`reconstruct_H`) does not work here either, since it bakes in one state’s `psi_st`, wrong for every other state.

$q(t)$ is instead available directly as a sparse diagonal matrix from `mf_potential.evaluate(t)`, with no probing required, so it is added straight into the batch’s right-hand side the same way `attach_fsys` adds the base drive’s own contribution, together with its own $-i q(t) \psi_{st}$ source term, embedded into the full (central plus absorbing-buffer) space and held frozen over each self-consistent window. This is implemented as `GPUSelfConsistentState`, a thin wrapper that mirrors `SelfConsistentState`’s update loop and wraps, rather than modifies, `BatchGPUSolver`’s right-hand-side function, leaving `gpu_solver_core.py` itself unchanged. Validated on the system in Table 4, matching native CPU `interaction.SelfConsistentState` to relative error 8×10^{-4} (reproduced: 8.155×10^{-4}). Both sides are sensitive to τ at this interaction strength: native CPU `tkwant`’s own trajectory disagrees with itself across $\tau = 1.0, 0.5, 0.2, 0.1$ until τ is small enough, a property of the feedback problem itself rather than of the GPU wrapper.

Table 4: Self-consistent validation system. Full definition: `test_gpu_selfconsistent.py`.

Lattice	<code>kwant.lattice.square(a=1, norbs=1)</code> , $L = 20$, $W = 1$ (1D)
Onsite, hopping	4, -1
Chemical potential	3
Drive	voltage pulse on lead 0: $0.3142(1 + \text{erf}((t - 50)/12.01))$
Mean field	diagonal onsite Hartree, interaction strength 2
Self-consistent step	$\tau = 0.1$
Evaluated at	$t = 10, 40, 80, 100$
n_{dof} , physical sites	197 (incl. CAP buffer), 20
N_{states}	8
GPU $H(t)$ update	non-monochromatic fallback, linear, $\Delta t_{\text{rc}} = 0.1$

One piece remains open: evaluating the mean-field operator’s expectation value at each self-consistent checkpoint. `tkwant`’s own quadrature-weighted `evaluate()` (lead-occupation weights, MPI-reduced) expects native per-state `onebody` objects, so the current implementation supplies this by syncing `psibar` back to those objects after every GPU-batched `evolve()` call, which is already how `BatchGPUSolver.evolve` ends, and then calling `evaluate()` on the CPU, amortized over a

self-consistent window rather than every ODE substep. A maintainer-exposed batched evaluation entry point would remove even that overhead, and is a concrete item worth raising if there is interest in collaborating further.

Two further upstream changes, neither required for this note’s results but each removing a specific piece of overhead or friction, are worth naming alongside the collaboration request itself. First, exposing H_0 on `kernels.Simple` the same way `kernels.Scipy` already does (Sec. 3) would remove the column-probing cost entirely, the 379s one-time reconstruction at production scale (Sec. 5). Second, letting `kernel_type` pass straight through `calc_initial_state/ScatteringStates` rather than requiring a nested `functools.partial` would make selecting the `kernels.Scipy` fast path less awkward for a `tkwant` user to opt into.

GPU-acceleration papers for an existing scientific code are a recognized category in venues such as Computer Physics Communications, and recent examples in adjacent sub-fields report multi-GPU scaling, which this note does not yet address (only a single device was available for this work). If there is interest, we would welcome the maintainers’ thoughts on developing this further, multi-GPU scaling included, toward a joint publication.

References

- [1] T. Kloss, J. Weston, R. Gaury, B. Rossignol, C. Groth, and X. Waintal, *Tkwant: a software package for time-dependent quantum transport*, New J. Phys. **23**, 023025 (2021).
- [2] C. W. Groth, M. Wimmer, A. R. Akhmerov, and X. Waintal, *Kwant: a software package for quantum transport*, New J. Phys. **16**, 063065 (2014).
- [3] J. Weston and X. Waintal, *Linear-scaling source-sink algorithm for simulating time-resolved quantum transport and superconductivity*, Phys. Rev. B **93**, 134506 (2016).
- [4] Y. Park, J. Sohn, S. Kim, and K.-W. Kim, *Electronic Orbital Angular Momentum Driven by Finite-Momentum Phonons: Beyond Chiral and Axial Phonons*, arXiv:2609.08100 (2026).
- [5] T. Kloss and X. Waintal, *Propagation of ultrashort voltage pulses through a small quantum dot*, Phys. Rev. B **111**, 235411 (2025).
- [6] S. Ouacel *et al.*, *Electronic interferometry with ultrashort plasmonic pulses*, arXiv:2408.13025 (2025).